

✓ Lab: Model Order Selection for Neural Data

Machine learning is a key tool for neuroscientists to understand how sensory and motor signals are encoded in the brain. In addition to improving our scientific understanding of neural phenomena, understanding neural encoding is critical for brain machine interfaces. In this lab, you will use model selection for performing some simple analysis on real neural signals.

Before doing this lab, you should review the ideas in the [polynomial model selection demo](#). In addition to the concepts in that demo, you will learn to:

- Represent neural time-series data in arrays
- Load data from a pickle file
- Describe and fit memoryless linear models
- Describe and fit linear time-series models with delays
- Fit linear models with multiple target outputs
- Select the optimal delay via cross-validation

✓ Loading the data

The data in this lab comes from neural recordings described in:

[Stevenson, Ian H., et al. "Statistical assessment of the stability of neural movement representations." *Journal of neurophysiology* 106.2 \(2011\): 764-774](#)

Neurons are the basic information processing units in the brain. Neurons communicate with one another via *spikes* or *action potentials* which are brief events where voltage in the neuron rapidly rises then falls. These spikes trigger the electro-chemical signals between one neuron and another. In this experiment, the spikes were recorded from 196 neurons in the primary motor cortex (M1) of a monkey using an electrode array implanted onto the surface of a monkey's brain. During the recording, the monkey performed several reaching tasks and the position and velocity of the hand was recorded as well.

The goal of the experiment is to try to *read the monkey's brain*: That is, predict the hand motion from the neural signals from the motor cortex.

We first load the key packages.

```
import numpy as np
import matplotlib.pyplot as plt
import pickle

from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score
```

The full data is available on the CRCNS website <http://crcns.org/data-sets/movements/dream>. This website has a large number of great datasets and can be used for projects as well. However, the raw data files can be quite large. To make the lab easier, the [Kording lab](#) at UPenn has put together an excellent [repository](#) where they have created simple pre-processed versions of the data. You can download the file `example_data_s1.pickle` from the [Dropbox link](#). Alternatively, you can directly run the following code. This may take a little while to download since the file is 26 MB.

```
fn_src = 'https://www.dropbox.com/sh/n4924ipcfjqc0t6/AAD0v9JYMUBK1t1lg9P71gSSra/example_da'
fn_dst = 'example_data_s1.pickle'

import os
from six.moves import urllib

if os.path.isfile(fn_dst):
    print('File %s is already downloaded' % fn_dst)
else:
    urllib.request.urlretrieve(fn_src, fn_dst)
```

The file is a *pickle* data structure, which is a package to serialize python objects into data files. Once you have downloaded the file, you can run the following command to retrieve the data from the pickle file.

```
with open('example_data_s1.pickle', 'rb') as fp:
    X,y = pickle.load(fp)
```

The matrix `X` is matrix of spike counts where `X[i,j]` is the number of spikes from neuron `j` in time bin `i`. The matrix `y` has two columns:

- `y[i,0]` = velocity of the monkey's hand in the x-direction
- `y[i,1]` = velocity of the monkey's hand in the y-direction Our goal will be to predict `y` from `X`.

Each time bin represent `tsamp=0.05` seconds of time. Using `X.shape` and `y.shape` compute and print:

- `nt =` the total number of time bins
- `nneuron =` the total number of neurons
- `nout =` the total number of output variables to track = number of columns in `y`
- `ttotal =` total time of the experiment is seconds.

```
tsamp = 0.05 # sampling time in seconds

# TODO 1
# nt = ...
# nneuron = ..
# nout = ...
# ttotal = ...
```

```
nt = X.shape[0]
nneuron = X.shape[1]
nout = y.shape[1]
ttotal = nt*tsamp
```

✓ Fitting a Memoryless Linear Model

Let's first try a simple linear regression model to fit the data.

First, use the `train_test_split` function to split the data into training and test. Let `Xtr,ytr` be the training data set and `Xts,yts` be the test data set. Use `test_size=0.33` so `1/3` of the data is used for test.

```
from sklearn.model_selection import train_test_split

# TODO 2
# Xtr, Xts, ytr, yts = ...
Xtr,Xts,ytr,yts = train_test_split(X,y,test_size = 0.33,random_state = 23)
```

Now, fit a linear model using `Xtr,ytr`. Make a prediction `yhat` using `Xts`. Compare `yhat` to `yts` to measure `rsq`, the R^2 . You can use the `r2_score` method. Print the `rsq` value. You should get `rsq` of around `0.45`.

```
# TODO 3
# yhat = ...
# rsq = ...
yhat = LinearRegression().fit(Xtr,ytr).predict(Xts)
rsq = r2_score(yts,yhat)
print(rsq)
```

```
0.4680550455260932
```

It is useful to plot the predicted vs. true values. Since we have two outputs, create two `subplots` using the `plt.subplot()` command. In plot `i=0,1`, plot `yhat[:,i]` vs. `yts[:,i]` with a scatter plot. Label the axes of the plots. You may also use the command:

```
plt.figure(figsize=(10,5))
```

to make the figures a little larger.

```
# TODO 4
plt.figure(figsize=(10,5))
```

```
<Figure size 1000x500 with 0 Axes>
<Figure size 1000x500 with 0 Axes>
```

✓ Fitting Models with Delay

One way we can improve the model accuracy is to use a delayed version of the features. Specifically, the model we used above mapped the features

$$\hat{y}[i,k] = \sum_{j=0}^{p-1} X[i,j] * w[j,k] + b[k]$$

where p is the number of features and $w[j,k]$ is a matrix of coefficients. In this model, $\hat{y}[i,:]$ at time i was only dependent on the inputs $X[i,:]$ at time i . In signal processing, this is called a *memoryless* model. However, in many physical systems, such as those that arise in neuroscience, there is a delay between the inputs $X[i,:]$ and the outputs $y[i]$. For such cases, we can use a model of the form,

$$\hat{y}[i+d,k] = \sum_{j=0}^{p-1} \sum_{m=0}^d X[i+m,j] * w[j,m,k] + b[k]$$

where w is a 3-dim array of coefficients where:

$w[j,m,k]$ is the influence of the input $X[i+m,j]$ onto output $y[i+d,k]$

In signal processing, this model is called an *FIR* filter and $w[j,:,k]$ is the *impulse response* from the j -th input to the k -th output. The point is that the output at time $i+d$ depends on the inputs at times $i, i+1, \dots, i+d$. Hence, it depends on the last $d+1$ time steps, not just the most recent time.

To translate this into a linear regression problem, complete the following function that creates a new feature and target matrix where:

$X_{dly}[i,:]$ has the rows $X[i,:]$, $X[i+1,:]$, ..., $X[i+dly,:]$
 $y_{dly}[i,:] = y[i+dly,:]$

Thus, $X_{dly}[i,:]$ contains all the delayed features for the target $\hat{y}$. Note that if X is $n \times p$ then X_{dly} will be $(n-dly) \times (dly+1)*p$.

```
def create_dly_data(X,y,dly):
    """
    Create delayed data
    """
    # TODO 5
    # Xdly = ...
    # ydly = ...

    n,p = X.shape

    Xdly = np.zeros((n-dly, (dly+1)*p))
    ydly = np.zeros((n-dly, y.shape[1]))

    for i in range(n - dly):
```

```

        Xdly[i,:] = X[i:i+dly+1,:].flatten()
        ydly[i,:] = y[i+dly,:]

    return Xdly, ydly

```

Now fit an linear delayed model with `dly=6` additional delay lags. That is,

- Create delayed data `Xdly,ydly=create_dly_data(X,y,dly=6)`
- Split the data into training and test as before
- Fit the model on the training data
- Measure the `R^2` score on the test data

If you did this correctly, you should get a new `R^2` score around 0.69. This is significantly better than the memoryless models.

```
# TODO 6
```

```

Xdly,ydly = create_dly_data(X,y,dly=6)
Xtr,Xts,ytr,yts = train_test_split(Xdly,ydly,test_size = 0.33,random_state = 23)
yhat = LinearRegression().fit(Xtr,ytr).predict(Xts)
rsq = r2_score(yts,yhat)
print(rsq)

```

```
0.6967918404248115
```

Plot the predicted vs. true values as before. You should visually see a better fit.

```
# TODO 7
```

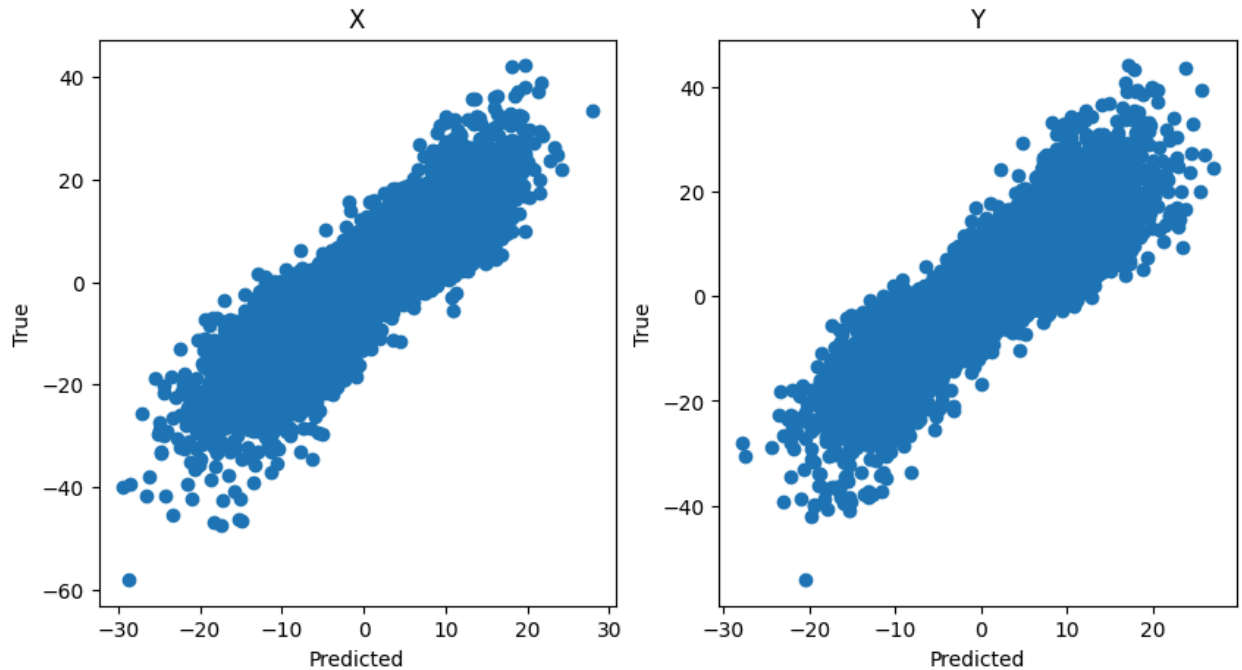
```

plt.figure(figsize=(10,5))
plt.subplot(1,2,1)
plt.scatter(yhat[:,0],yts[:,0])
plt.xlabel('Predicted')
plt.ylabel('True')
plt.title('X')

plt.subplot(1,2,2)
plt.scatter(yhat[:,1],yts[:,1])
plt.xlabel('Predicted')
plt.ylabel('True')
plt.title('Y')

```

Text(0.5, 1.0, 'Y')



Note: Fitting an FIR model with the above method is very inefficient when the number of delays, `dly` is large. In the above method, the number of columns of `X` grows from `p` to `(dly+1)*p` and the computations become expensive with `dly` is large. We will describe a much faster way to fit such models using gradient descent when we talk about convolutional neural networks.

✓ Selecting the Optimal Delay via Model Order Selection

In the previous example, we fixed `dly=6`. We can now select the optimal delay using model order selection. Since we have a large number of data samples, it turns out that the optimal model order uses a very high delay. Using the above fitting method, the computations take too long. So, to simplify the lab, we will first just pretend that we have a very limited data set.

Compute `Xred` and `yred` by taking the first `nred=6000` samples of the data `X` and `y`. This is about 10% of the overall data.

```
nred = 6000

# TODO 8
# Xred = ...
# yred = ...

Xred = X[:nred,:]
yred = y[:nred,:]
```

Now complete the following code to implement K-fold cross validation with `nfold=5` and values of delays `dtest = [0,1,...,dmax]`.

The code also includes a progress bar using the `tqdm` package. This is very useful when you have a long computation.

Note: Some students appeared to use the `mse` metric (i.e. RSS per sample) instead of `R^2`. That is fine. For the solution, I have computed both.

You may have an issue with tqdm, make sure you have the necessary tools installed: Check https://ipywidgets.readthedocs.io/en/stable/user_install.html

```
import sklearn.model_selection
import tqdm.notebook

nfold = 5 # Number of folds
dmax = 15 # maximum number of delays

# TODO 9: Create a k-fold object
# kf = sklearn.model_selection.KFold(n_splits=..., shuffle=True, random_state=23)
kf = sklearn.model_selection.KFold(n_splits=nfold, shuffle=True, random_state=23)

# TODO 10: Model orders to be tested
# dtest = vector [0,1,...,dmax]
# nd = len(dtest)
dtest = np.arange(dmax+1)
nd = len(dtest)

# TODO 11.
# Initialize a matrix Rsq to hold values of the R^2 across the model orders and folds.
# Alternatively, you can also create an RSS matrix
Rsq = np.zeros((nd,nfold))

# Create a progress bar. Note there are nd*nfold total fits.
pbar = tqdm.notebook.tqdm(
    total=nfold*nd, initial=0,
    unit='fits', unit_divisor=nd, desc='Model order test')

for it, d in enumerate(dtest):
    # TODO 12:
    # Create the delayed data using the create_dly_function from the reduced
    # data Xred, yred
    # Xdly, ydly = create_dly_data(...)
    Xdly, ydly = create_dly_data(Xred, yred, d)

    # Loop over the folds
    for isplit, Ind in enumerate(kf.split(Xdly)):

        # Get the training data in the split
        Itr, Its = Ind

        # TODO 13
        # Split the data (Xdly,ydly) into training and test
```

```

# Xtr = ...
# ytr = ...
# Xts = ...
# yts = ...
Xtr = Xdly[Itr,:]
ytr = ydly[Itr,:]
Xts = Xdly[Its,:]
yts = ydly[Its,:]

# TODO 14: Fit data on training data
model = LinearRegression().fit(Xtr, ytr)

# TODO 15: Measure the R^2 value on test data and store in the matrix Rsq
yhat = model.predict(Xts)
Rsq[it,isplit] = r2_score(yts,yhat)
pbar.update(1)
pbar.close()

```

Model order test: 100%

80/80 [00:47<00:00, 1.15fits/s]

Compute the mean and standard error of the R^2 values as a function of the model order d . Use a `plt.errorbar` plot. Label your axes.

```

from numpy._core.fromnumeric import mean
# TODO 16
mean_Rsq = np.mean(Rsq,axis=1)
std_Rsq = np.std(Rsq,axis=1,ddof=1)/np.sqrt(nfold)
plt.errorbar(dtest,mean_Rsq,yerr=std_Rsq)
plt.xlabel('Delay')
plt.ylabel('R^2')

```

Text(0, 0.5, 'R^2')

Find the optimal order d with the normal rule (i.e. highest test R^2)

```
# TODO 17
d = np.argmax(mean_Rsq)
print(d)
```

8 0.60

Now find the optimal model order via the one SE rule (i.e. the lowest order model which has test R^2 score within one SE)

```
# TODO 18
threshold = mean_Rsq[d] - std_Rsq[d]
d = np.where(mean_Rsq >= threshold)[0][0]
print(d)
```

6 0.45

Start coding or [generate](#) with AI.